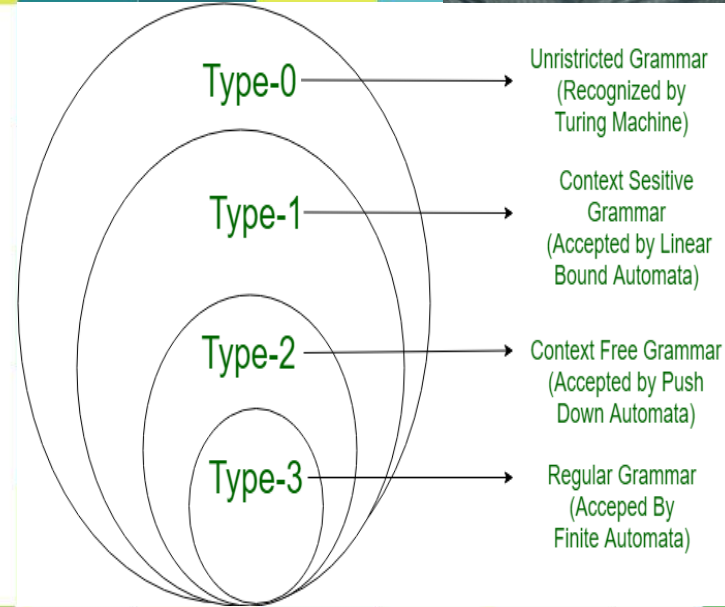




BCSC0011: THEORY OF AUTOMATA & FORMAL LANGUAGES

(Module-II)



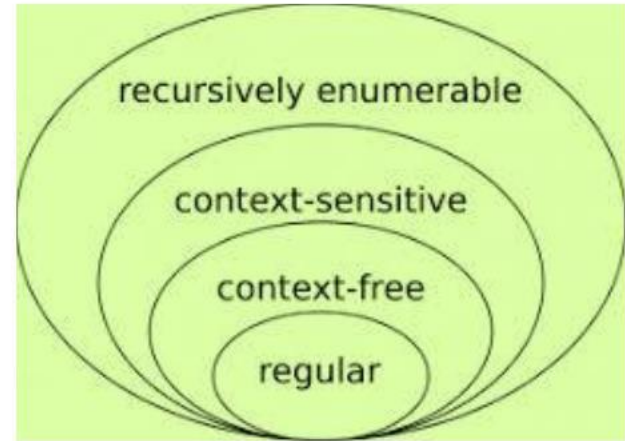
By: Dr. Sandeep Rathor

Module II

- Context Free Grammar and Context Free Languages: Introduction, Derivation Trees or Parse Tree
- Ambiguity in Grammar, Ambiguous to Unambiguous CFG
- Simplification of CFGs:** Removal of useless symbols, elimination of null productions, elimination of unit productions
- Normal Forms** for CFGs - CNF and GNF
- Pumping lemma for CFLs
- Equivalence of PDA and CFG
- Turing Machine**

Context-Free Grammars

- Languages that are **generated** by context-free grammars are context-free languages
- Context-free grammars are more expressive than finite automata: if a language L is **accepted** by a finite automata then L can be **generated** by a context-free grammar
- Beware:
The converse is NOT true



A Grammar is the collection of 4-tuple i.e.

$G = (V_N, \Sigma, P, S)$ where:

V_N : Set of Variables; **Σ :** Set of Terminals

P : Production Rule; **S :** Start Symbol

$V_N = \{ \langle \text{Sentence} \rangle, \langle \text{noun} \rangle, \langle \text{verb} \rangle, \langle \text{adverb} \rangle \}$

$\Sigma = \{ \text{sandeep, hari, read, quickly} \}$

$S = \langle \text{Sentence} \rangle$

$\langle \text{Sentence} \rangle \quad \langle \text{noun} \rangle \langle \text{verb} \rangle \langle \text{adverb} \rangle$

$\langle \text{noun} \rangle \longrightarrow \text{Sandeep, hari}$

$\langle \text{verb} \rangle \longrightarrow \text{read}$

$\langle \text{adverb} \rangle \longrightarrow \text{quickly}$



Validation Rules:

1. If $S \rightarrow BC$ is a given production, it means we can replace S by BC but can not replace BC by S . **[Reverse substitution is not possible]**
2. If $S \rightarrow BC$ is a given production, it is not necessary that $BC \rightarrow S$ is a production. **[No inverse operation is permitted]**



Formal Definition Grammar

$$G = (V_N, \Sigma, P, S)$$

Finite Set of
Variables /
Non
Terminals

Finite Set of
terminal
symbols

Set of
Productions rules

Start
variable

All productions in **P** are of
the form

$$A \rightarrow \alpha$$

Variable

String of
variables and
terminals

Example of Context-Free Grammar

$$S \rightarrow aSb \mid \lambda$$

$$P = \{S \rightarrow aSb, S \rightarrow \lambda\}$$

productions

$$G = (V_N, \Sigma, S, P)$$

$$V_N = \{S\}$$

variables

$$\Sigma = \{a, b\}$$

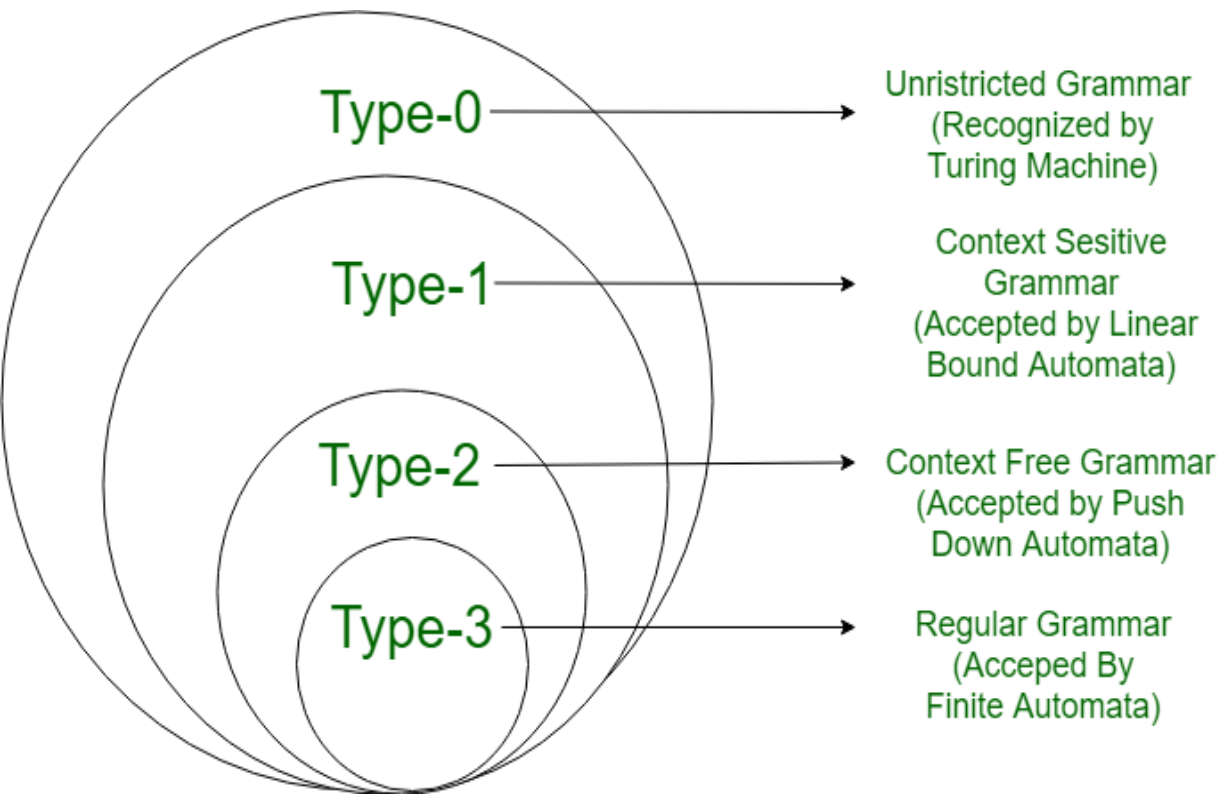
terminals

start variable

Chomsky Classification on Grammar

According to Noam Chomsky, there are four types of grammars – Type 0, Type 1, Type 2, and Type 3.

Grammar Type	Grammar Accepted	Language Accepted	Automaton
Type 0	Unrestricted grammar	Recursively enumerable language	Turing Machine
Type 1	Context-sensitive grammar	Context-sensitive language	Linear-bounded automaton
Type 2	Context-free grammar	Context-free language	Pushdown automaton
Type 3	Regular grammar	Regular language	Finite state automaton



Type 0 known as unrestricted grammar.
Type 1 known as context sensitive grammar.
Type 2 known as context free grammar.
Type 3 Regular Grammar.

Type-0 grammars

The productions have no restrictions.

Type-1 grammars generate context-sensitive languages. The productions must be in the form $\alpha A \beta \rightarrow \alpha \gamma \beta$ where $A \in N$ (Non-terminal) and $\alpha, \beta, \gamma \in (V_N \cup \Sigma)^*$ (Strings of terminals and non-terminals)

The strings α and β may be empty, but γ must be non-empty.

$AB \rightarrow AbBc$
 $A \rightarrow bcA$
 $B \rightarrow b$

Type-2 grammars generate context-free languages.

The productions must be in the form $A \rightarrow \alpha$, where $A \in V_N$ (Non terminal) and $\alpha \in (V_N \cup \Sigma)^*$

It should be type1

$S \rightarrow Xa$

$X \rightarrow a$

$X \rightarrow aX$

$X \rightarrow abc$

$X \rightarrow \epsilon$

Type-3 grammars generate regular languages.

Type-3 grammars must have a single non-terminal on the left-hand side and a right-hand side consisting of a single terminal or single terminal followed by a single

The productions must be in the form $X \rightarrow a$ or $X \rightarrow aY$

where $X, Y \in V_N$ (Non terminal)

and $a \in \Sigma$ (Terminal)

$X \rightarrow \epsilon$

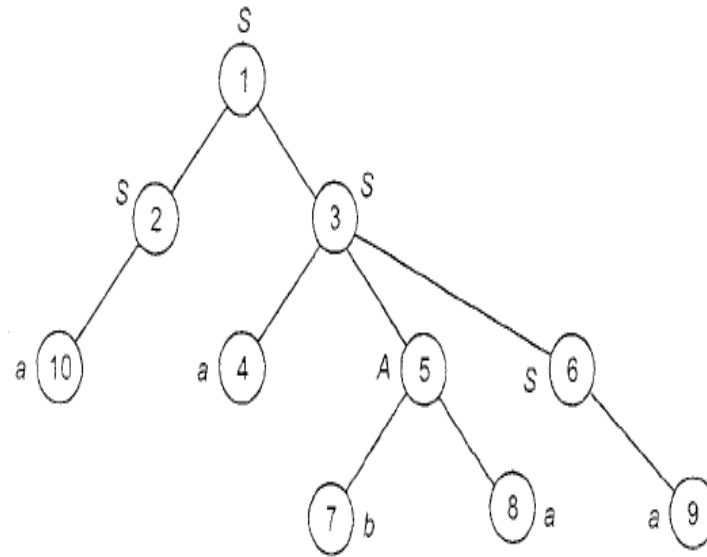
$X \rightarrow a \mid aY$

$Y \rightarrow b$

Derivation Trees or Parse Tree

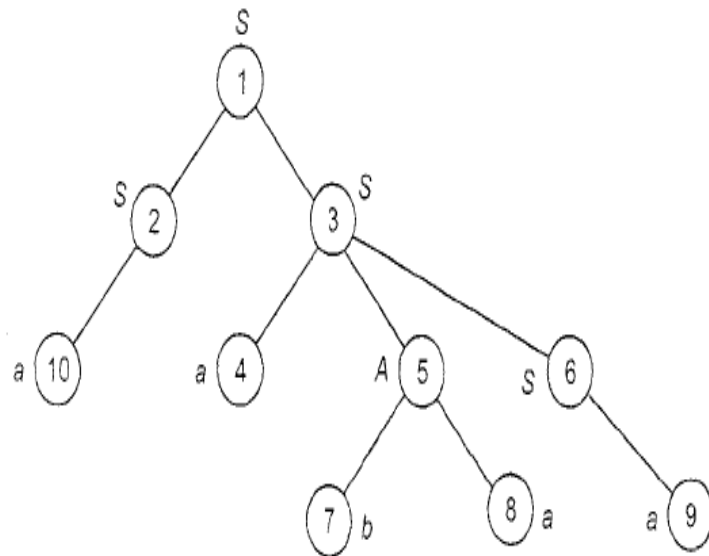
- Given a derivable string, we may represent the derivation by a derivation / parse tree.
- A **parse tree** is a tree with the following properties:
 - Every vertex has a label which is a variable or terminal or null (λ).
 - The root node is the start symbol S .
 - The label of an internal vertex is a variable.
 - If the vertices $n_1, n_2, \dots, n_k$ written with labels $X_1, X_2, \dots, X_k$ are the sons of vertex n with label A , then $A \rightarrow X_1, X_2, \dots, X_k$ is a production in P .
 - A vertex n is a leaf if its label is $a \in \Sigma$ or null.

Example: Let $G = (\{S, A\}, \{a, b\}, P, S)$, where P consists of $S \rightarrow aAS \mid a \mid SS$, $A \rightarrow SbA \mid ba$



Note: The yield of a derivation tree is the concatenation of the labels of the leaves without repetition in the left to right ordering

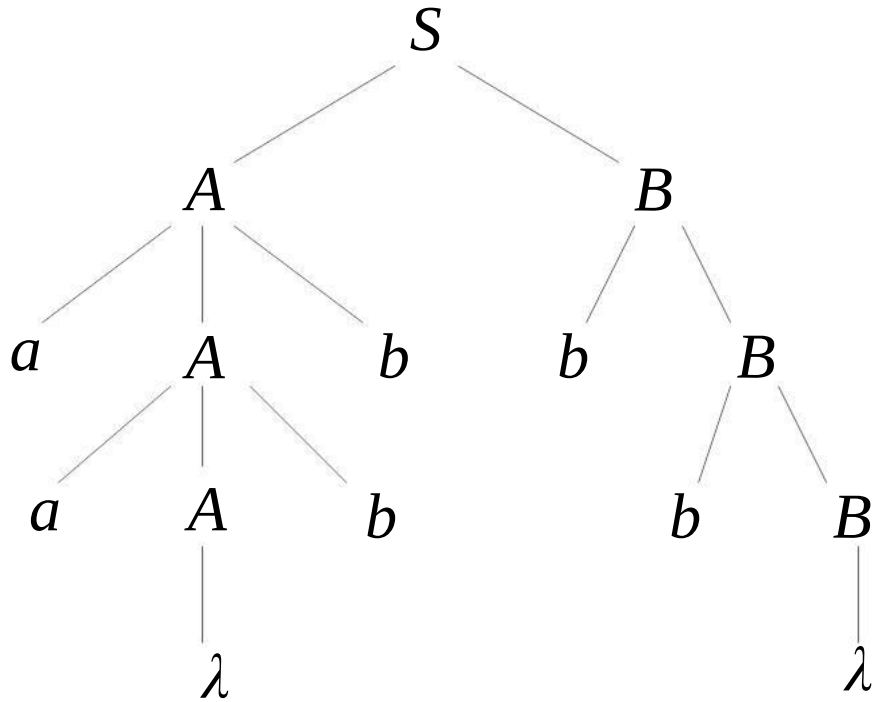
Example: Let $G = (\{S, A\}, \{a, b\}, P, S)$, where P consists of $S \rightarrow aAS \mid a \mid SS$, $A \rightarrow SbA \mid ba$. Find derivation tree for string **aabaa**



Example of a Derivation Tree

- Let the grammar G be
 - $S \rightarrow AB$
 - $A \rightarrow aAb \mid \lambda$
 - $B \rightarrow bB \mid \lambda$
- Let the string be $w = aabbbb$.
- Derive the string w .

Now draw the Parse Tree of *aabbbb*



This is the parse tree of many different, but equivalent, derivations of *aabbbb*.

For example,

$S \Rightarrow AB \Rightarrow aAbB \Rightarrow aaAbbB \Rightarrow aabbB \Rightarrow aabbbbB \Rightarrow aabbbb$.

$S \Rightarrow AB \Rightarrow AbB \Rightarrow AbbB \Rightarrow Abb \Rightarrow aAbbb \Rightarrow aaAbbbb \Rightarrow aabbbb$.

Leftmost and Rightmost Derivations

1. Leftmost Derivations

- A *leftmost derivation* of a string is a derivation in which each production rule is applied to the leftmost non terminal in the string.

or

A derivation $A \Rightarrow^* w$ is called a *left most derivation* (LMD) if we apply a production only to the leftmost variable at every step.

2. Rightmost Derivations

- A *rightmost derivation* of a string is a derivation in which each production rule is applied to the rightmost non terminal in the string.

or

A derivation $A \xRightarrow{*} w$ is called a *rightmost derivation* (RMD) if we apply a production only to the rightmost variable at every step.

Example: Let G be the grammar $S \rightarrow 0B \mid 1A$, $A \rightarrow 0 \mid 0S \mid 1AA$, $B \rightarrow 1 \mid 1S \mid 0BB$. For the string 00110101, find:

- a) LMD b) RMD c) Derivation Tree or Parse Tree

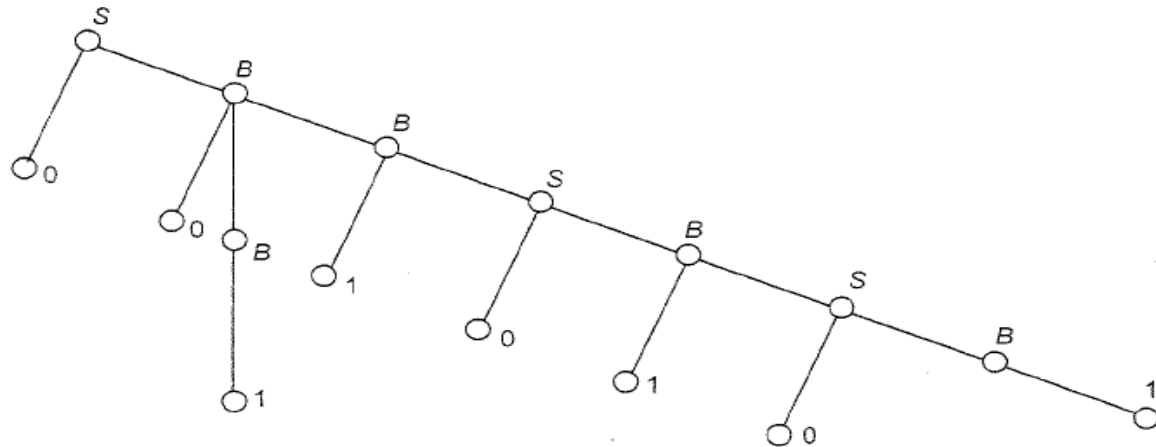
Solution: LMD:

$S \Rightarrow 0B \Rightarrow 00\textcolor{red}{B} \Rightarrow 001B \Rightarrow 0011S \Rightarrow 00110B \Rightarrow 001101S \Rightarrow 0011010B \Rightarrow 00110101$

RMD:

$S \Rightarrow 0B \Rightarrow 00B\textcolor{red}{B} \Rightarrow 00B1\textcolor{red}{S} \Rightarrow 00B10\textcolor{red}{B} \Rightarrow 00B101\textcolor{red}{S} \Rightarrow 00B1010\textcolor{red}{B} \Rightarrow 00\textcolor{red}{B}10101 \Rightarrow 00110101$

Parse Tree:



Ambiguous Grammars

- ▼ If there exists $w \in L(G)$ such that w has two or more different
 - parse trees or
 - leftmost (or rightmost) derivations ,
- ▼ then G is *ambiguous*.

Consider the grammar G with production rules –

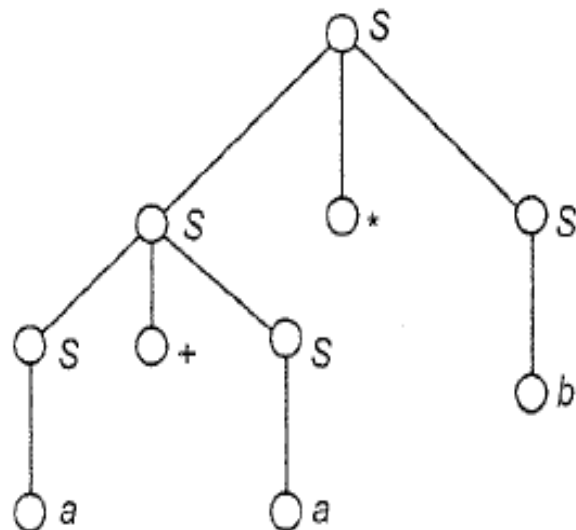
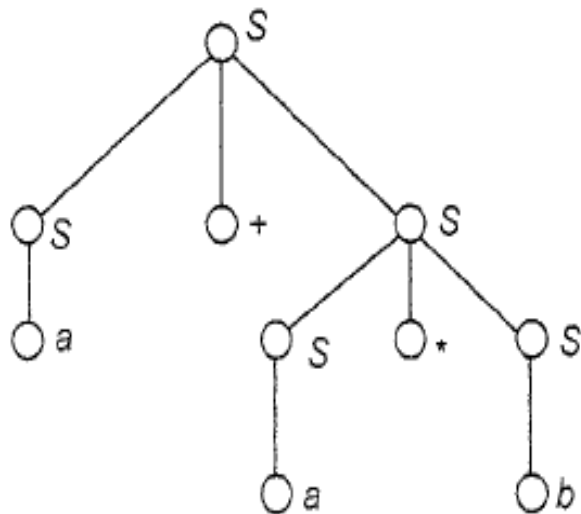
$$S \rightarrow S+S \mid S*S \mid a \mid b$$

Find two Left Most Derivations of the string "**a+a*b**"

Derivation 1 : $S \rightarrow S+S \rightarrow a+S \rightarrow a+S*S \rightarrow a+a*S \rightarrow a+a*b$

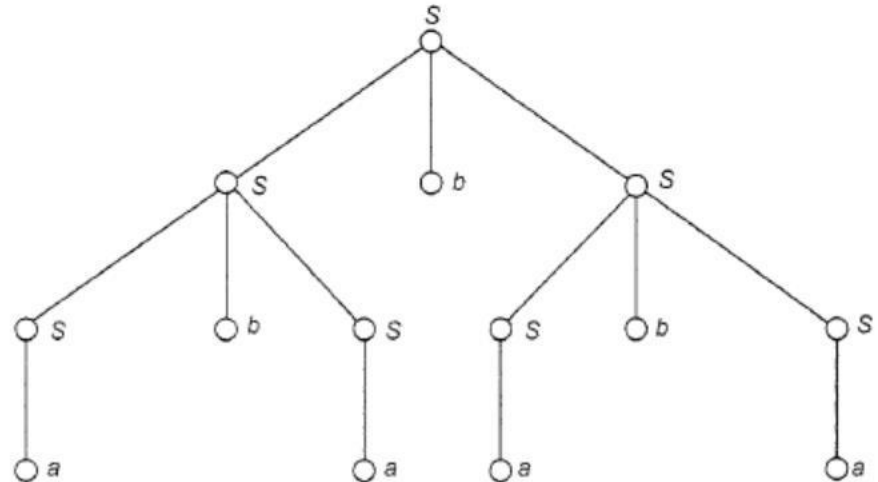
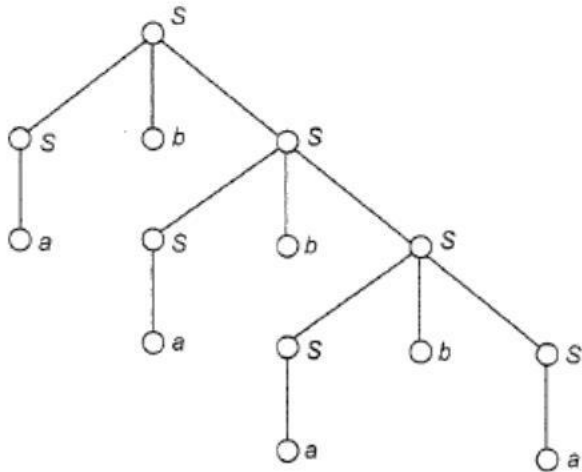
Derivation 2 : $S \rightarrow S*S \rightarrow S+S*S \rightarrow a+S*S \rightarrow a+a*S \rightarrow a+a*b$

Parse trees for **a+a*b**



If G is the grammar $S \rightarrow SbS \mid a$, show that G is ambiguous.

Consider $w=abababa$



Example1

$E \rightarrow E + E$

$E \rightarrow E - E$

$E \rightarrow id$

String "id + id - id"

First Leftmost derivation

$E \rightarrow E + E$

$\rightarrow id + E$

$\rightarrow id + E - E$

$\rightarrow id + id - E$

$\rightarrow id + id - id$

Second Leftmost derivation

$E \rightarrow E - E$

$\rightarrow E + E - E$

$\rightarrow id + E - E$

$\rightarrow id + id - E$

$\rightarrow id + id - id$

Prove that grammar
is ambiguous...

Example1

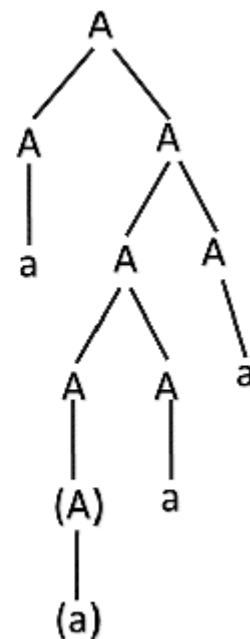
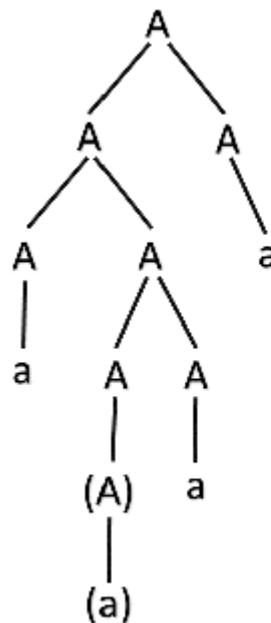
$A \rightarrow AA$

$A \rightarrow (A)$

$A \rightarrow a$

For the string "a(a)aa"

Two different parse tree for the same string:



- Ambiguity can be removed by rewriting the grammar such that there is only one derivation or parse tree possible for a string of the language which the grammar represents.

E.g.

- ▶ Ambiguous Grammar

$S \rightarrow SbS \mid a$

- ▶ Can be rewritten as

$S \rightarrow abS \mid a$

There are some inherently ambiguous languages like
 $L = \{a^n b^n c^m\} \cup \{a^n b^m c^m\}$



Rule: $A \rightarrow A\alpha_1 \mid A\alpha_2 \mid A\alpha_3 \mid \dots \mid A\alpha_m \mid B_1 \mid B_2 \mid \dots \mid B_n$ where no B_i begin with A .
then we replace the A -production by:

$A \rightarrow B_1 A' \mid B_2 A' \mid \dots \mid B_n A'$

$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \alpha_m A' \mid \lambda$

[Note: productions of the form of $A \rightarrow A\alpha$, called immediate left recursion]

Example: $E \rightarrow E+T \mid T$, $T \rightarrow T*F \mid F$, $F \rightarrow (E) \mid a$

Solution:

$E \rightarrow TE'$ $E' \rightarrow +TE' \mid \lambda$

$T \rightarrow FT'$ $T' \rightarrow *FT' \mid \lambda$

$F \rightarrow (E) \mid a$

Simplification of CFGs



Can't always eliminate ambiguity.

But, CFG simplification & restriction still useful
theoretically & pragmatically.

- Simpler grammars are easier to understand.
- Simpler grammars can lead to faster parsing.
- Restricted forms useful for some parsing algorithms.
- Restricted forms can give you more knowledge about derivations.

■ Context Free Grammar can be simplified by:

■ Useless production elimination

or

Construction of Reduced Grammar

■ ϵ production elimination

■ Unit production elimination

Useless Productions

- A non-terminal X is **useless** if:

- X does not generate any string of terminals.
 - It cannot derive a terminal string

OR

- X does not occur in any sentential form
 - It cannot be reached from start symbol

Derive some strings

$$S \rightarrow aSb \mid c \mid Ac$$

$$A \rightarrow aA$$

$$B \rightarrow bB \mid b$$

Any production involving a useless symbol is a **useless production**.

Elimination of Useless Productions

Phase 1 – Derivation of an equivalent grammar, G' , from the CFG, G , such that **each variable derives some terminal string**.

Construction of V' :

- ♦ We define $V_i' \subseteq V$ by recursion
- ♦ **$V_1' = \{A \in V \mid \text{there exists a production } A \rightarrow \omega \text{ where } \omega \in \Sigma^*\}$**
- ♦ $V_{i+1}' = V_i' \cup \{A \in V \mid \text{there exists some production } A \rightarrow \alpha \text{ with } \alpha \in (\Sigma \cup V_i')^*\}$
- ♦ At some point **$V_k' = V_{k+1}'$** . Then we get $V' = V_k'$
- ♦ Construction of P'
- ♦ $P' = \{A \rightarrow \alpha \mid A, \alpha \in (V' \cup \Sigma)^*\}$

Let G be $\mathbf{S \rightarrow AB, A \rightarrow a, B \rightarrow b, B \rightarrow C, \text{ and } E \rightarrow c}$
Find G' such that every variable in G' derives some terminal string.

$$V_1' = \{A, B, E\}$$

$$A \rightarrow a, B \rightarrow b, E \rightarrow c$$

$$V_2' = \{A, B, E, S\}$$

$$S \rightarrow AB$$

$$V_3' = \{A, B, E, S\}$$

$$= V_2'$$

Here we get $V' = \{S, A, B, E\}$.

So $P' = S \rightarrow AB, A \rightarrow a, B \rightarrow b$ and $E \rightarrow c$

Phase 2 – Derivation of an equivalent grammar, G'' , from the CFG, G' , such that **each symbol appears in a sentential form**.

Construction of V'' :

- ♦ We define $V_i'' \subseteq V$ by recursion
- ♦ $V_1'' = \{S\}$
- ♦ $V_{i+1}'' = V_i'' \cup \{X \in V' \mid \text{there exists some production } A \rightarrow \alpha \text{ with } A \in V_i'' \text{ \& } \alpha \text{ contains the symbol } X\}$
- ♦ At some point $V_k'' = V_{k+1}''$. Then we get $V'' = V_k''$
- ♦ Construction of P''
- ♦ $P'' = \{A \rightarrow \alpha \mid A, \alpha \in (V'' \cup \Sigma)^*\}$

We had **G** as **$S \rightarrow AB$** , **$A \rightarrow a$** , **$B \rightarrow b$** , **$B \rightarrow C$** , and **$E \rightarrow c$**

Then we got **G'** as **$S \rightarrow AB$** , **$A \rightarrow a$** , **$B \rightarrow b$** and **$E \rightarrow c$**

$$V_1'' = \{S\}$$

$$V_2'' = \{S, A, B\}$$

$$S \rightarrow AB$$

$$V_3'' = \{S, A, B\}$$

$$A \rightarrow a, B \rightarrow b$$

$$= V_2''$$

Here we get $V'' = \{S, A, B\}$.

So $P'' = S \rightarrow AB, A \rightarrow a, B \rightarrow b$

Question: Find a reduced grammar equivalent to the grammar G, having production rules, $P: S \rightarrow AC \mid B, A \rightarrow a, C \rightarrow c \mid BC, E \rightarrow aA \mid e$.

Solution:

Step1:

Terminals = $\{ a, c, e \}$

$W_1 = \{ A, C, E \}$ from rules $A \rightarrow a, C \rightarrow c$ and $E \rightarrow e$

$W_2 = \{ A, C, E \} \cup \{ S \}$ from rule $S \rightarrow AC$

$W_3 = \{ A, C, E, S \} \cup \emptyset$

Since $W_2 = W_3$, we can derive G' as –

$G' = \{ \{ A, C, E, S \}, \{ a, c, e \}, P, \{ S \} \}$, where $P: S \rightarrow AC, A \rightarrow a, C \rightarrow c, E \rightarrow aA \mid e$

Step2:

$V_1 = \{ S \}$

$V_2 = \{ S, A, C \}$ from rule $S \rightarrow AC$

$V_3 = \{ S, A, C, a, c \}$ from rules $A \rightarrow a$ and $C \rightarrow c$

$V_4 = \{ S, A, C, a, c \}$

Since $V_3 = V_4$, we can derive G'' as –

$G'' = \{ \{ A, C, S \}, \{ a, c \}, P, \{ S \} \}$, where $P: S \rightarrow AC, A \rightarrow a, C \rightarrow c$

Construct a reduced grammar equivalent to the grammar:

$S \rightarrow aAa, \quad A \rightarrow Sb \mid bCC \mid DaA, \quad C \rightarrow abb \mid DD,$

$E \rightarrow aC, \quad D \rightarrow aDA$

Solution: Step1:

$W1 = \{C\}$ as $C \rightarrow abb$ is the only production with a terminal string on the R.H.S.

$W2 = \{C\} \cup \{E, A\}$, as $E \rightarrow aC$ and $A \rightarrow bCC$ are productions with R.H.S. in $(\Sigma \cup \{C\})^*$

$W3 = \{C, E, A\} \cup \{S\}$, as $S \rightarrow aAa$ and aAa is in $(\Sigma \cup W2)^*$

$W4 = W3 \cup \emptyset$

Hence, $V_n = W = \{S, A, C, E\}$

$P' = \{S \rightarrow aAa, A \rightarrow Sb \mid bCC, C \rightarrow abb, E \rightarrow aC\}$

$G' = (V_n, \{a, b\}, P', S)$

Solution: Step2:

$W1 = \{S\}$, As we have $S \rightarrow aAa$,

$W2 = \{S\} \cup \{A, a\}$, As $A \rightarrow Sb \mid bCC$,

$W3 = \{S, A, a\} \cup \{S, b, C\} = \{S, A, C, a, b\}$

As we have $C \rightarrow abb$,

$W4 = W3 \cup \{a, b\} = W3$

Hence $P'' = \{S \rightarrow aAa, \quad A \rightarrow Sb \mid bCC, \quad C \rightarrow abb\}$

$G'' = (\{S, A, C\}, \{a, b\}, P'', S)$ is the reduced grammar.

Question: Find a reduced grammar equivalent to the grammar G whose productions are
 $S \rightarrow AB \mid CA$, $B \rightarrow BC \mid AB$, $A \rightarrow a$, $C \rightarrow aB \mid b$

Solution:

Step1:

$G' = (\{S, A, C\}, \{a, b\}, \{S \rightarrow CA, A \rightarrow a, C \rightarrow b\}, S)$

Step2:

$G'' = (\{S, A, C\}, \{a, b\}, \{S \rightarrow CA, A \rightarrow a, C \rightarrow b\}, S)$

Elimination of Null Productions

- A CFG may have productions of the form $A \rightarrow \lambda$. So, a production in the form of $A \rightarrow \lambda$, where A is variable, is *c/d null production*.
- If G is a context free grammar, then we can find a context free grammar G_1 having no null productions such that $L(G) = L(G_1) \cup \{\lambda\}$

Step 1: Construction of the set W of all nullable variables

- ▶ $W_1 = \{A_1 \in V \mid A_1 \rightarrow \lambda \text{ is a production in } P\}$
- ▶ $W_{i+1} = W_i \cup \{K \in V \mid \exists \text{ a production } K \rightarrow \alpha \text{ with } \alpha \in W_i^* \}$
- ▶ At some point $W_{i+1} = W_i$

$$S \rightarrow aS \mid AB$$

$$A \rightarrow \lambda \mid a$$

$$B \rightarrow \lambda \mid b$$

$$D \rightarrow b$$

$$W_1 = \{A, B\}$$

$$W_2 = \{A, B, S\}$$

$$W_3 = \{A, B, S\} \\ = W_2$$

Step 2: Construction P'

- ▶ All prodⁿs whose RHS doesn't have any nullable variables are included in P'

- ▶ If $A \rightarrow X_1 X_2 \dots X_k$ is in P then include

$$A \rightarrow \alpha_1 \alpha_2 \dots \alpha_k \quad \text{where}$$

$$\alpha_i = \begin{cases} X_i & \text{if } X_i \notin W \\ \lambda \text{ or } X_i & \text{if } X_i \in W \end{cases}$$

$$\text{Provided } A \rightarrow \alpha_1 \alpha_2 \dots \alpha_k \neq \lambda$$

$$A \rightarrow a$$

$$B \rightarrow b$$

$$D \rightarrow b$$

$$S \rightarrow aS \mid a \mid AB \mid A \mid B$$

Question: Remove null production from the following –
 $S \rightarrow ASA \mid aB \mid b, A \rightarrow B, B \rightarrow b \mid \epsilon$

Solution:

$S \rightarrow ASA \mid aB \mid b \mid a \mid SA \mid AS \mid S, \quad A \rightarrow B, \quad B \rightarrow b$

Eliminate Null productions:

1. $S \rightarrow ABAC$

$A \rightarrow aA / \epsilon$

$B \rightarrow bB / \epsilon$

$C \rightarrow c$

Solution after null elimination:

$S \rightarrow ABAC / ABC / BAC / BC / AAC / AC / C$

$A \rightarrow aA / a$

$B \rightarrow bB / b$

$C \rightarrow c$

Eliminate Null productions:

2. $S \rightarrow aSb / aAb / ab / a$

$A \rightarrow \epsilon$

Solution after null elimination:

$S \rightarrow aSb / aAb / ab / a$

Eliminate Null(ϵ)-Productions

1. $S \rightarrow AB$, $A \rightarrow aAA/\epsilon$, $B \rightarrow bBB/\epsilon$

Solution:

$S \rightarrow AB | A | B$,

$A \rightarrow aAA | aA | a$

$B \rightarrow bBB | bB | b$

2. $S \rightarrow ABCd$

$A \rightarrow BC$, $B \rightarrow bB \mid \epsilon$, $C \rightarrow cC \mid \epsilon$

Solution:

$S \rightarrow ABCd \mid ABd \mid ACd \mid BCd \mid Ad \mid Bd \mid Cd \mid d$

$A \rightarrow BC \mid B \mid C$

$B \rightarrow bB \mid b$

$C \rightarrow cC \mid c$

Eliminate Null(ϵ)-Productions

1. $S \rightarrow ABA$, $A \rightarrow aA/\epsilon$, $B \rightarrow bB/\epsilon$

Solution:

$S \rightarrow ABA \mid AB \mid BA \mid AA \mid A \mid B$

$A \rightarrow aA \mid a$

$B \rightarrow bB \mid b$

2. $S \rightarrow ABCd$

$A \rightarrow BC$, $B \rightarrow bB \mid \epsilon$, $C \rightarrow cC \mid \epsilon$

Solution:

$S \rightarrow ABCd \mid ABd \mid ACd \mid BCd \mid Ad \mid Bd \mid Cd \mid d$

$A \rightarrow BC \mid B \mid C$

$B \rightarrow bB \mid b$

$C \rightarrow cC \mid c$

Eliminate Null Productions

$S \rightarrow ABCBCDA$

$A \rightarrow CD$

$B \rightarrow Cb$

$C \rightarrow a \mid \lambda$

$D \rightarrow bD \mid \lambda$

The CFG will have total 40
prodⁿs!!

➤ 32 S prodⁿs

➤ $A \rightarrow CD \mid C \mid D$

➤ $B \rightarrow Cb \mid b$

➤ $C \rightarrow a$

➤ $D \rightarrow bD \mid b$

Elimination of Unit Productions

$A \rightarrow B$ where $A, B \in V$

Step 1: Construction of the set variables, derivable from A

- ▶ $W_1(A) = \{A\}$
- ▶ $W_{i+1}(A) = W_i(A) \cup \{C \in V \mid \text{there is a production } B \rightarrow C \text{ \& } B \in W_i(A)\}$
- ▶ At some point $W_{i+1}(A) = W_i(A)$

$S \rightarrow A \mid bb$

$A \rightarrow B \mid b$

$B \rightarrow S \mid a$

$W(S) = \{S, A, B\}$

$W(A) = \{S, A, B\}$

$W(B) = \{S, A, B\}$

$W_1(S) = \{S\}$

$W_2(S) = \{S, A\}$

$W_3(S) = \{S, A, B\} = W_4(S)$

$W(S) = \{S, A, B\}$

Step 2: All non unit prodⁿs are included as such

$$S \rightarrow bb$$

$$A \rightarrow b$$

$$B \rightarrow a$$

Step 3: Add prodⁿ $A \rightarrow \alpha$ whenever

- ▶ $B \in W(A)$ and
- ▶ $B \rightarrow \alpha$ is in P and
- ▶ $\alpha \notin V$

$$S \rightarrow A \mid bb$$

$$A \rightarrow B \mid b$$

$$B \rightarrow S \mid a$$

$$W(S) = \{S, A, B\}$$

$$W(A) = \{S, A, B\}$$

$$W(B) = \{S, A, B\}$$

$$S \rightarrow b \mid a$$

$$A \rightarrow bb \mid a$$

$$B \rightarrow bb \mid b$$

Hence prodⁿs in G' are:

$$S \rightarrow a \mid b \mid bb$$

$$A \rightarrow a \mid b \mid bb$$

$$B \rightarrow a \mid b \mid bb$$

Eliminate Unit-productions

$S \rightarrow Aa \mid B$

$A \rightarrow b \mid B$

$B \rightarrow A \mid a$

Solution:

$S \rightarrow Aa \mid b \mid a$

$A \rightarrow b \mid a$

$B \rightarrow a \mid b$

B-production doesn't occur in the production 'S', then the following grammar becomes

$S \rightarrow Aa \mid b \mid a$

$A \rightarrow b \mid a$

Question: Remove unit production from the following:

$S \rightarrow XY, X \rightarrow a, Y \rightarrow Z \mid b, Z \rightarrow M, M \rightarrow N, N \rightarrow a$

Solution:

$S \rightarrow XY, X \rightarrow a, Y \rightarrow a \mid b, Z \rightarrow a, M \rightarrow a, N \rightarrow a$

Now Z, M, and N are unreachable, hence we can remove those.

The final CFG is unit production free –

$S \rightarrow XY, X \rightarrow a, Y \rightarrow a \mid b$

Eliminates unit production:

$E \rightarrow E + T \mid T, \quad T \rightarrow T * F \mid F, \quad F \rightarrow (E) \mid a$

Solution after Elimination of unit production:

$E \rightarrow E + T \mid T * F \mid (E) \mid a$

$T \rightarrow T * F \mid (E) \mid a$

$F \rightarrow (E) \mid a$

Simplify the following grammar:

$S \rightarrow 0A0 \mid 1B1 \mid BB,$ $A \rightarrow C,$ $B \rightarrow S \mid A$ $C \rightarrow S \mid \epsilon$

Solution after null removal:

$S \rightarrow 0A0 \mid 00 \mid 1B1 \mid 11 \mid B \mid BB$

$A \rightarrow C$

$B \rightarrow S \mid A$

$C \rightarrow S$

Solution without unit production:

$S \rightarrow 0A0 \mid 00 \mid 1B1 \mid 11 \mid BB$

$A \rightarrow 0A0 \mid 00 \mid 1B1 \mid 11 \mid BB$

$B \rightarrow 0A0 \mid 00 \mid 1B1 \mid 11 \mid BB$

$C \rightarrow 0A0 \mid 00 \mid 1B1 \mid 11 \mid BB$

Simplified Grammar:

Symbol C is not reachable i.e. useless

so final simplified grammar:

$S \rightarrow 0A0 \mid 00 \mid 1B1 \mid 11 \mid BB$

$A \rightarrow 0A0 \mid 00 \mid 1B1 \mid 11 \mid BB$

$B \rightarrow 0A0 \mid 00 \mid 1B1 \mid 11 \mid BB$

Simplify the following grammar:

$S \rightarrow AB \mid BC \mid aACb \mid a$, $A \rightarrow AAB \mid BD \mid abD \mid C$, $C \rightarrow CA \mid S \mid a$, $D \rightarrow d$, $E \rightarrow ab$

*Answer is not verified, pls verify it.

Solution without unit production:

$S \rightarrow AB \mid BC \mid aACb \mid a$

$A \rightarrow AAB \mid BD \mid abD \mid CA \mid AB \mid BC \mid aACb \mid a$

$C \rightarrow CA \mid AB \mid BC \mid aACb \mid a$

$D \rightarrow d$, $E \rightarrow ab$

Simplified Grammar:

Symbol E is not reachable i.e. useless so

final simplified grammar:

$S \rightarrow AB \mid BC \mid aACb \mid a$

$A \rightarrow AAB \mid BD \mid abD \mid CA \mid AB \mid BC$
 $\mid aACb \mid a$

$C \rightarrow CA \mid AB \mid BC \mid aACb \mid a$

$D \rightarrow d$

Normal Forms

▣ Chomsky Normal Form (CNF)

▣ Griebach Normal Form (GNF)

By:
Dr. Sandeep Rathor

Chomsky Normal Form

- A CFG is in CNF if all the prodⁿs are in any one of the following forms
 - $A \rightarrow a$
 - $A \rightarrow BC$where A , B , and C are non-terminals and a is a terminal

Steps of Chomsky Normal Form

Step1: Elimination of null production and unit production

Step2: Elimination of terminals on RHS

Step3: Restricting the number of variable on RHS

Convert the following CFG into CNF

$$S \rightarrow aSa \mid bSb \mid aa \mid bb \mid a \mid b$$

▣ No null or unit prodⁿs or $\varepsilon \in L$

▣ Add prodⁿs already in CNF

▣ Convert $S \rightarrow aSa$

▣ Convert $S \rightarrow bSb$

▣ Convert $S \rightarrow aa$

▣ Convert $S \rightarrow bb$

$$S \rightarrow a \mid b$$

$$S \rightarrow N_a S N_a, \quad N_a \rightarrow a$$

$$S \rightarrow N_a S_1$$

$$S_1 \rightarrow S N_a$$

$$S \rightarrow N_b S N_b, \quad N_b \rightarrow b$$

$$S \rightarrow N_b S_2$$

$$S_2 \rightarrow S N_b$$

$$S \rightarrow N_a N_a$$

$$S \rightarrow N_b N_b$$

Convert the following CFG into CNF

$$S \rightarrow aAD, A \rightarrow aB | bAB, B \rightarrow b, D \rightarrow d$$

▢ No null or unit prodⁿs

▢ Add prodⁿs already in CNF

$$B \rightarrow b, D \rightarrow d$$

$S \rightarrow aAD$ gives rise to $S \rightarrow CaAD$ and $Ca \rightarrow a$.

$A \rightarrow aB$ gives rise to $A \rightarrow CaB$.

$A \rightarrow bAB$ gives rise to $A \rightarrow CbAB$ and $Cb \rightarrow b$.

$$V' = \{S, A, B, D, Ca, Cb\}.$$

$P1$ consists of $S \rightarrow CaAD$, $A \rightarrow CaB | CbAB$,
 $B \rightarrow b$, $D \rightarrow d$, $Ca \rightarrow a$, $Cb \rightarrow b$.

$A \rightarrow CaB$, $B \rightarrow b$, $D \rightarrow d$, $Ca \rightarrow a$, $Cb \rightarrow b$ are
added to $P2$

$S \rightarrow CaAD$ is replaced by $S \rightarrow CaC1$ and $C1 \rightarrow AD$.

$A \rightarrow CbAB$ is replaced by $A \rightarrow CbC2$ and $C2 \rightarrow AB$.

Let

$$G2 = (\{S, A, B, D, Ca, Cb, C1, C2\}, \{a, b, d\}, P2, S)$$

where $P2$ consists of $S \rightarrow CaC1$, $A \rightarrow CaB | CbC2$,

$C1 \rightarrow AD$, $C2 \rightarrow AB$, $B \rightarrow b$, $D \rightarrow d$, $Ca \rightarrow a$.

$Cb \rightarrow b$. $G2$ is in CNF and equivalent to G .

Practice Session

Question1: $S \rightarrow aAbB$, $A \rightarrow aA \mid a$, $B \rightarrow bB \mid b$. Convert it into CNF

Solution:

$G = (\{S, A, B, Ca, Cb, C1, C2\}, \{a, b\}, P2, S)$,

where $P2$ consists of $S \rightarrow CaC1$, $C1 \rightarrow AC2$, $C2 \rightarrow CbB$, $A \rightarrow CaA$,
 $B \rightarrow CbB$, $Ca \rightarrow a$, $Cb \rightarrow b$, $A \rightarrow a$ and $B \rightarrow b$.

Question2:

Find a grammar in CNF equivalent to the grammar

$$S \rightarrow \sim S \mid [S \supset S] \mid p \mid q \quad (S \text{ being the only variable})$$

$$G_2 = (\{S, A, B, C, D, C_1, C_2, C_3\}, \Sigma, P_2, S)$$

where P_2 consists of $S \rightarrow p \mid q \mid AS \mid BC_1$, $A \rightarrow \sim$, $B \rightarrow [$, $C \rightarrow \supset$, $D \rightarrow]$,
 $C_1 \rightarrow SC_2$, $C_2 \rightarrow CC_3$, $C_3 \rightarrow SD$. G_2 is in CNF and equivalent to the given grammar.

Find the CNF:

$S \rightarrow a \mid aA \mid B$

$A \rightarrow aBB \mid \epsilon$

$B \rightarrow Aa \mid b$

Solution:

$S \rightarrow a \mid XA \mid AX \mid b$

$A \rightarrow RB$

$B \rightarrow AX \mid b \mid a$

$X \rightarrow a$

$R \rightarrow XB$

Greibach Normal Form

- A context-free grammar is in Greibach Normal Form if the right-hand side of each rule has one terminal followed by zero or more non-terminals:

$$A \rightarrow a \alpha$$

where

$$a \in \Sigma$$

$$\alpha \in V^*$$

Example:

$$S \rightarrow aB \mid bA$$

$$A \rightarrow a \mid aS \mid bAA$$

$$B \rightarrow b \mid bS \mid aBB$$

Finally A2-productions are:

$$A2 \rightarrow aA1 \mid b \mid aA1Z2 \mid bZ2 \quad (1)$$

$$A1 \rightarrow A2A2 \mid a$$

Apply lemma1 to A1-production

A1 $\rightarrow$ A2A2 then we find

$$A1 \rightarrow aA1A2 \mid bA2 \mid aA1Z2A2 \mid bZ2A2$$

Finally A1- productions are:

$$A1 \rightarrow a \mid aA1A2 \mid bA2 \mid aA1Z2A2 \mid bZ2A2 \quad (2)$$

Z2-productions are:

$$Z2 \rightarrow A2A1, Z2 \rightarrow A2A1Z2, \text{ [Apply lemma1]}$$

$$\begin{aligned} Z2 &\rightarrow aA1A1 \mid bA1 \mid aA1Z2A1 \mid bZ2A1 \\ Z2 &\rightarrow aA1A1Z2 \mid bA1Z2 \mid aA1Z2A1Z2 \mid bZ2A1Z2 \end{aligned} \quad (3)$$

Equivalent grammar is:

$G' = (\{A1, A2, Z2\}, \{a, b\}, P1, A1)$ where P1 shown as:
(1)(2) and (3).

Convert given CFG to GNF

$S \rightarrow AB$

$A \rightarrow BS \mid b$

$B \rightarrow SA \mid a$

Step 1 & 2: Convert to CNF

Already in CNF

Step 3: Rename the variables & Rewrite the prodⁿs

$S - A_1 \quad A - A_2 \quad B - A_3$

$A_1 \rightarrow A_2 A_3$

$A_2 \rightarrow A_3 A_1 \mid b$

$A_3 \rightarrow A_1 A_2 \mid a$

Step 4: Check for Loop

$A_3 \rightarrow A_1 A_2 \quad // i > j$

$A_3 \rightarrow A_2 A_3 A_2 \quad // i > j$

$A_3 \rightarrow A_3 A_1 A_3 A_2 \mid b A_3 A_2$

Step 5: Eliminate Left Recursion

$A_1 \rightarrow A_2 A_3$

$A_2 \rightarrow A_3 A_1 \mid b$

$A_3 \rightarrow A_3 A_1 A_3 A_2 \mid b A_3 A_2 \mid a$

Convert into GNF

$S \rightarrow XB \mid AA$

$A \rightarrow a \mid SA$

$B \rightarrow b$

$X \rightarrow a$

Solution:

$S \rightarrow aB \mid aCA \mid aBACA \mid aA \mid aBAA$

$A \rightarrow aC \mid aBAC \mid a \mid aBA$

$C \rightarrow aCAC \mid aBACAC \mid aAC \mid aBAAC$

$C \rightarrow aCA \mid aBACA \mid aA \mid aBAA$

$B \rightarrow b$

$X \rightarrow a$

Find GNF of the given CFG:

$E \rightarrow E + T \mid T, \quad T \rightarrow T * F \mid F, \quad F \rightarrow (E) \mid a$

Questions for Practice

1. Convert following grammar into CNF:

$$S \rightarrow bA / aB$$

$$A \rightarrow bAA / aS / a$$

$$B \rightarrow aBB / bS / b$$

2. Convert following grammar to PDA:

$$S \rightarrow aAA$$

$$A \rightarrow aS / bS / a$$

3. Eliminate Null and Unit production:

$$S \rightarrow aXbX$$

$$X \rightarrow aY / bY / \wedge$$

$$Y \rightarrow X / c$$

After Null

$$S \rightarrow aXbX / abX / aXb / ab$$

$$X \rightarrow aY / bY / a / b$$

$$Y \rightarrow X / c$$

After Unit

$$S \rightarrow aXbX / abX / aXb / ab$$

$$X \rightarrow aY / bY / a / b$$

$$Y \rightarrow aY / bY / a / b / c$$

Discussion

Eliminate null production:

$S \rightarrow ASA \mid aB \mid b, A \rightarrow B, B \rightarrow b \mid \epsilon$

Solution:

$S \rightarrow ASA \mid aB \mid b \mid a \mid SA \mid AS \mid S, A \rightarrow B \mid b, B \rightarrow b$

Remove unit production:

$S \rightarrow XY, X \rightarrow a, Y \rightarrow Z \mid b, Z \rightarrow M, M \rightarrow N, N \rightarrow a$

Solution:

$S \rightarrow XY, X \rightarrow a, Y \rightarrow a \mid b, Z \rightarrow a, M \rightarrow a, N \rightarrow a$ (after removal of unit production)

$S \rightarrow XY, X \rightarrow a, Y \rightarrow a \mid b$ [after removal of unreachable symbols since, Z, M, and N are unreachable]

Eliminate null production:

$S \rightarrow XYX$

$X \rightarrow 0X \mid \epsilon$

$Y \rightarrow 1Y \mid \epsilon$

Solution:

$S \rightarrow XY \mid YX \mid XX \mid X \mid Y$

$X \rightarrow 0X \mid 0$

$Y \rightarrow 1Y \mid 1$

Equivalence of Push Down Automata & Context Free Languages

By:
Dr. Sandeep Rathor

CFG to PDA

- ◆ The PDA simulates the **left-sentential forms** that G uses to generate any string w
- ◆ Let $G = (V, \Sigma, P, S)$
- ◆ Construct PDA N that **accepts $L(G)$ by empty stack**
- ◆ **$N = (\{q\}, \Sigma, V \cup \Sigma, \delta, q, S)$**
- ◆ Transitions are defined as

- For each variable A

$$\delta(q, \epsilon, A) = \{(q, \beta) \mid A \rightarrow \beta \text{ is a prod}^n \text{ in } G\}$$

- For each terminal a

$$\delta(q, a, a) = \{(q, \epsilon)\}$$

Find a PDA equivalent to the grammar **$S \rightarrow aSbb \mid a$**
Generate the string aabb and also simulate the PDA for it

◆ **$N = (\{q\}, \Sigma, V \cup \Sigma, \delta, q, S)$**

Where $\Sigma = \{a, b\}$, $V = \{S\}$ & δ is
as following

1. $\delta(q, \varepsilon, S) = \{(q, aSbb), (q, a)\}$
2. $\delta(q, a, a) = \{(q, \varepsilon)\}$
3. $\delta(q, b, b) = \{(q, \varepsilon)\}$

$S \Rightarrow aSbb \Rightarrow aabb$

(q, aabb, S)
├ (q, aabb, aSbb)
├ (q, abb, Sbb)
├ (q, abb, abb)
├ (q, bb, bb)
├ (q, b, b)
└ (q, ε , ε)

Stack Empty – string
accepted

Find a PDA equivalent to the following grammar that generates simple expressions like $a * (a + b010)$

$$\begin{aligned} E &\rightarrow I \mid E * E \mid E + E \mid (E) \\ I &\rightarrow a \mid b \mid Ia \mid Ib \mid Io \mid I1 \end{aligned}$$

Let N be a PDA that accepts the language generated by the given CFG

- ◆ $N = (\{q\}, \Sigma, V \cup \Sigma, \delta, q, S)$
- ◆ $\Sigma = \{a, b, o, 1, (,), *, +\}$
- ◆ $V = \{E, I\}$

1. $\delta(q, \varepsilon, E) = \{(q, I), (q, E^*E), (q, E+E), (q, (E))\}$
2. $\delta(q, \varepsilon, I) = \{(q, a), (q, b), (q, Ia), (q, Ib), (q, Io), (q, I1)\}$
3. $\delta(q, a, a) = \{(q, \varepsilon)\}$
4. $\delta(q, b, b) = \{(q, \varepsilon)\}$
5. $\delta(q, o, o) = \{(q, \varepsilon)\}$
6. $\delta(q, 1, 1) = \{(q, \varepsilon)\}$
7. $\delta(q, (, () = \{(q, \varepsilon)\}$
8. $\delta(q,),)) = \{(q, \varepsilon)\}$
9. $\delta(q, *, *) = \{(q, \varepsilon)\}$
10. $\delta(q, +, +) = \{(q, \varepsilon)\}$

PDA to CFG

We construct grammar G as follows.

$$G = (V, \Sigma, P, S)$$

$$\boxed{\text{▣}} \quad V = \{S\} \cup \{q, z, q' \mid q, q' \in Q, z \in \Gamma\}$$

The productions in P are induced by the moves of PDA as follows

$\boxed{\text{▣}}$ **Rule 1:**

The S productions are given by

$$S \rightarrow [q_o, Z_o, q] \quad \forall q \in Q$$

▣ Rule 2:

Each move erasing a pushdown symbol given by $(q', \varepsilon) \in \delta(q, a, z)$ induces the prodⁿ

$$[q, z, q'] \rightarrow a$$

▣ Rule 3:

Each move not erasing a pushdown symbol given by $(q_1, \underline{z_1 z_2 \dots z_m}) \in \delta(q, a, z)$ induces many prodⁿs of the following form where $q', q_2, \dots, q_m$ can be any state in Q

$$[q, z, q'] \rightarrow a [q_1, z_1, q_2] [q_2, z_2, q_3] \dots [q_m, z_m, q']$$

Construct a context-free grammar G which accepts $L(N)$, where

$$N = (\{q_0, q_1\}, \{a, b\}, \{Z_0, Z\}, \delta, q_0, Z_0, \varphi)$$

δ is given by

1. $\delta(q_0, b, Z_0) = \{(q_0, ZZ_0)\}$
2. $\delta(q_0, \varepsilon, Z_0) = \{(q_0, \varepsilon)\}$
3. $\delta(q_0, b, Z) = \{(q_0, ZZ)\}$
4. $\delta(q_0, a, Z) = \{(q_1, Z)\}$
5. $\delta(q_1, b, Z) = \{(q_1, \varepsilon)\}$
6. $\delta(q_1, a, Z_0) = \{(q_0, Z_0)\}$

Let $G = \{V, \{a, b\}, P, S\}$

$$\boxed{\square} \quad \mathbf{V = \{S\} \cup \{q, z, q'\} \mid q, q' \in Q, z \in \Gamma}$$

$$V = \{S\} \cup \{[q_0, Z_0, q_0], [q_0, Z_0, q_1],$$

$$[q_0, Z_1, q_0], [q_0, Z_1, q_1],$$

$$[q_1, Z_0, q_0], [q_1, Z_0, q_1],$$

$$[q_1, Z_1, q_0], [q_1, Z_1, q_1],$$

The productions in P are induced by the moves of PDA

▣ **Rule 1:** $S \rightarrow [q_o, Z_o, q] \forall q \in Q$

1. $S \rightarrow [q_o, Z_o, q_o]$

2. $S \rightarrow [q_o, Z_o, q_1]$

▣ **Rule 2:** $\delta(\mathbf{q}, \mathbf{a}, \mathbf{z}) = (\mathbf{q}', \varepsilon)$ induces $[\mathbf{q}, \mathbf{z}, \mathbf{q}'] \rightarrow \mathbf{a}$

- $\delta(q_0, \varepsilon, Z_0) = \{(q_0, \varepsilon)\}$

3. $[q_0, Z_0, q_0] \rightarrow \varepsilon$

- $\delta(q_1, b, Z) = \{(q_1, \varepsilon)\}$

4. $[q_1, Z, q_1] \rightarrow b$

▣ **Rule 3:** $\delta(q, a, z) = (q_1, z_1 z_2 \dots z_m)$

$[q, z, q'] \rightarrow a [q_1, z_1, q_2] [q_2, z_2, q_3] \dots [q_m, z_m, q']$

- $\delta(q_0, b, Z_0) = \{(q_0, ZZ_0)\}$

$[q_0, Z_0, \blacklozenge] \rightarrow b [q_0, Z, \blacklozenge] [\blacklozenge, Z_0, \blacklozenge]$

5. $[q_0, Z_0, q_0] \rightarrow b [q_0, Z, q_0] [q_0, Z_0, q_0]$
6. $[q_0, Z_0, q_0] \rightarrow b [q_0, Z, q_1] [q_1, Z_0, q_0]$
7. $[q_0, Z_0, q_1] \rightarrow b [q_0, Z, q_0] [q_0, Z_0, q_1]$
8. $[q_0, Z_0, q_1] \rightarrow b [q_0, Z, q_1] [q_1, Z_0, q_1]$

- $\delta(q_0, b, Z) = \{(q_0, ZZ)\}$

$$[q_0, Z, \blacklozenge] \rightarrow b [q_0, Z, \blacklozenge] [\blacklozenge, Z, \blacklozenge]$$

$$9. [q_0, Z, q_0] \rightarrow b [q_0, Z, q_0] [q_0, Z, q_0]$$

$$10. [q_0, Z, q_0] \rightarrow b [q_0, Z, q_1] [q_1, Z, q_0]$$

$$11. [q_0, Z, q_1] \rightarrow b [q_0, Z, q_0] [q_0, Z, q_1]$$

$$12. [q_0, Z, q_1] \rightarrow b [q_0, Z, q_1] [q_1, Z, q_1]$$

- $\delta(q_0, a, Z) = \{(q_1, Z)\}$

$$[q_0, Z, \blacklozenge] \rightarrow a [q_1, Z, \blacklozenge]$$

$$13. [q_0, Z, q_0] \rightarrow a [q_1, Z, q_0]$$

$$14. [q_0, Z, q_1] \rightarrow a [q_1, Z, q_1]$$

- $\delta(q_1, a, Z_0) = \{(q_0, Z_0)\}$

$$[q_1, Z_0, \blacklozenge] \rightarrow a [q_0, Z_0, \blacklozenge]$$

$$15. [q_1, Z_0, q_0] \rightarrow a [q_0, Z_0, q_0]$$

$$16. [q_1, Z_0, q_1] \rightarrow a [q_0, Z_0, q_1]$$

4. Convert following grammar into CNF:

$$S \rightarrow bA / aB$$

$$A \rightarrow bAA / aS / a$$

$$B \rightarrow aBB / bS / b$$

5. Convert following grammar to PDA:

$$S \rightarrow aAA$$

$$A \rightarrow aS / bS / a$$

6. Eliminate Null and Unit production:

$$S \rightarrow aXbX$$

$$X \rightarrow aY / bY / \wedge$$

$$Y \rightarrow X / c$$

After Null

$$S \rightarrow aXbX / abX / aXb / ab$$

$$X \rightarrow aY / bY / a / b$$

$$Y \rightarrow X / c$$

After Unit

$$S \rightarrow aXbX / abX / aXb / ab$$

$$X \rightarrow aY / bY / a / b$$

$$Y \rightarrow aY / bY / a / b / c$$